

Distributed Systems + Parallelism

Flow for today

- The scaling problem: data and work outgrow one machine
- Distributed data: partitioning, replication, and consistency trade-offs
- Distributed work: parallelism, divide-and-conquer, and bottlenecks
- Failure modes and decision checklist

Key terms: SLA

SLA (Service Level Agreement)

A commitment between provider and customer (e.g. “99.9% uptime” or “p95 response time < 200 ms”).



Key terms: Why we care about response-time distribution

When we measure **response time**
(e.g. how long a query or request takes),
we care about the full distribution,
not only the average.



Key terms: Latency percentiles (p50, p95, p99)

Term	Meaning	Why it matters
p50 (median)	Half of requests finish within this time	Typical user experience
p95	95% of requests finish within this time	Most user, long tail starts here
p99	99% of requests finish within this time	Worst regular users, SLA and penalties often defined here

Think like a system designer, not a programmer

- **Distributed database decisions**

Where is the data stored so service stays reliable and responsive?

- **Parallel processing decisions**

How is the workload split so decisions arrive on time at acceptable cost?

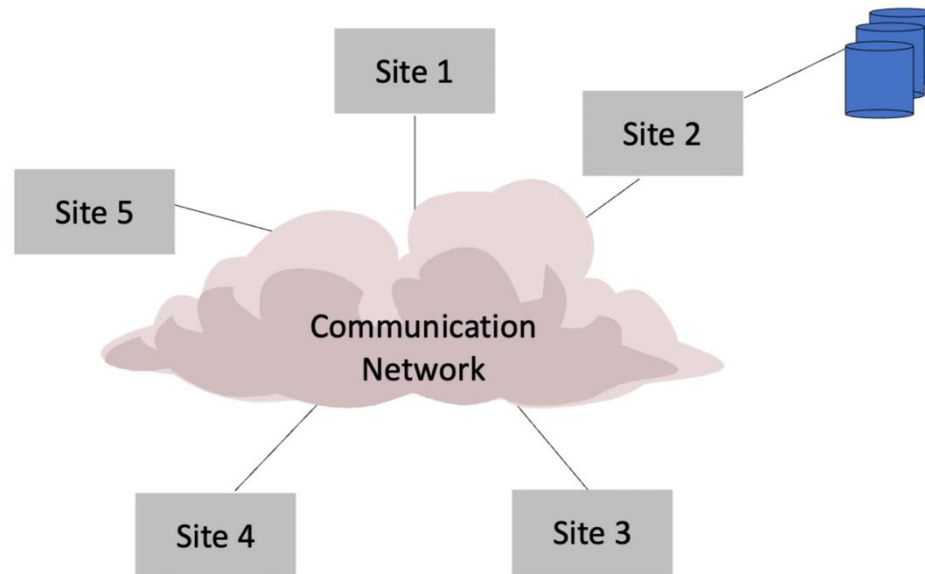
Two sides of the same coin

- **Distributed data**
many nodes store one logical database
(where the data lives)
- **Parallel compute**
many workers process one logical job
(how the work is split)

Same user experience, higher scale and resilience

The Scaling Problem (Business View)

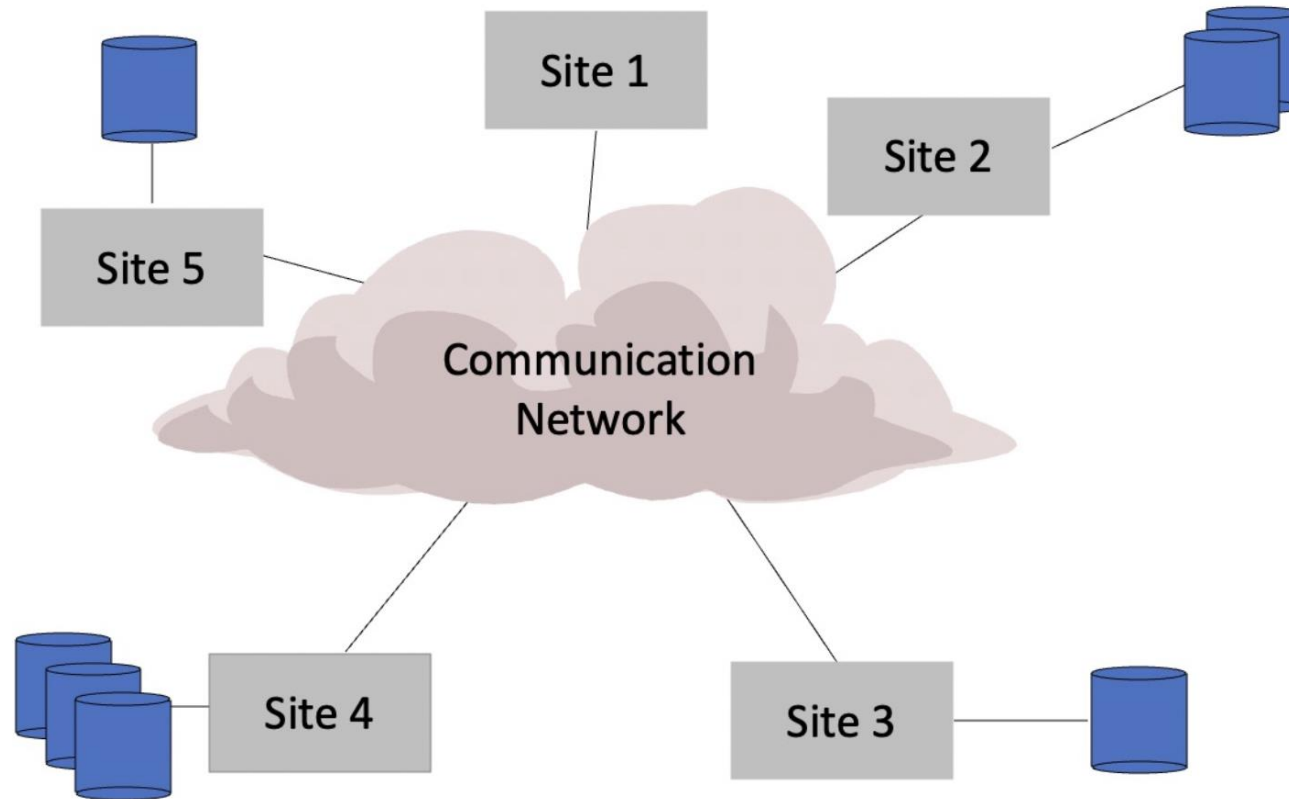
A single-node architecture often works early, then fails suddenly during growth.



Why this breaks (operations terms)

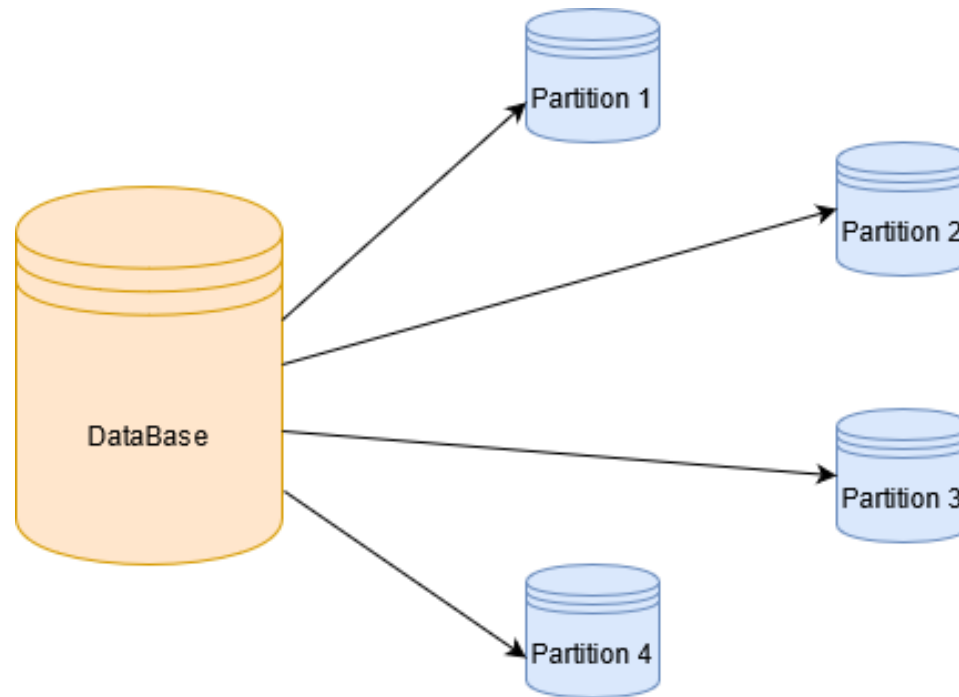
- One machine becomes a **bottleneck station**.
- Maintenance or failure creates a **single point of stoppage**.
- Peak demand creates long tails (p95/p99 explode before average does).

Distributed Databases



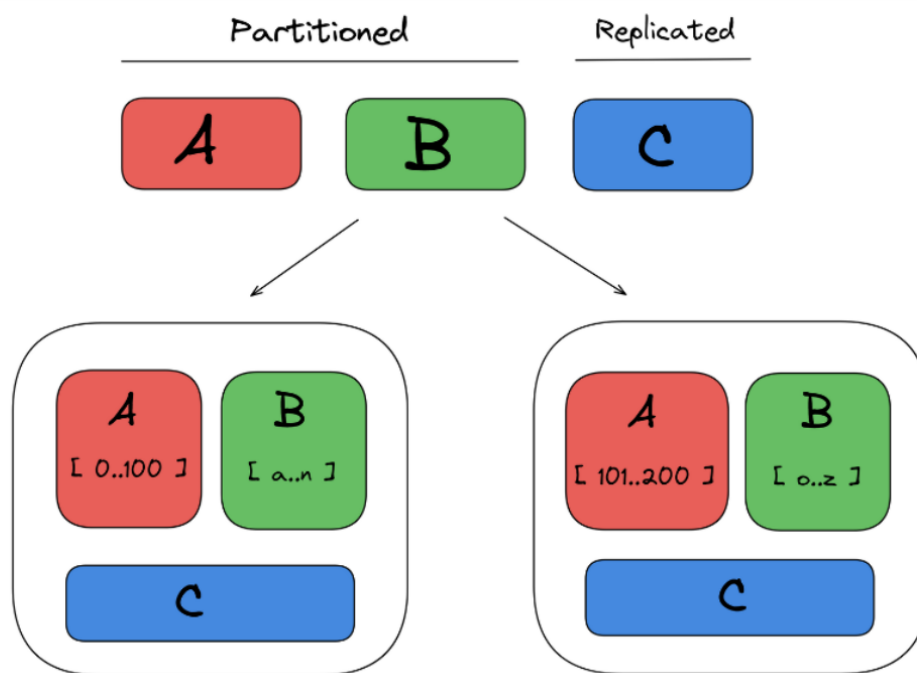
Partitioning: split data across nodes

Partitioning means dividing rows so each node stores a subset.



Replication: copy data for availability and read scale

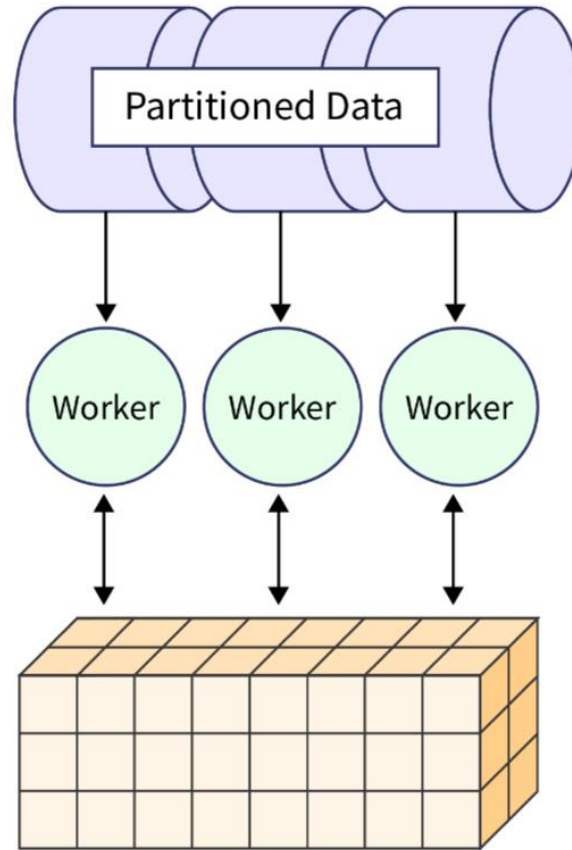
Replication keeps multiple copies (primary + replicas).



SQL vs NoSQL trade-offs

Dimension	SQL systems (typical)	NoSQL systems (typical)
Strength	Strong integrity, joins, transactions	Flexible schema
Best fit	Finance, orders, master data	Event logs, feeds, session/state
Cost pattern	Higher coordination cost at very high scale	Lower per-operation coordination
Risk pattern	Performance bottlenecks under extreme scale	Consistency/freshness caveats

Parallelism & Work Distribution



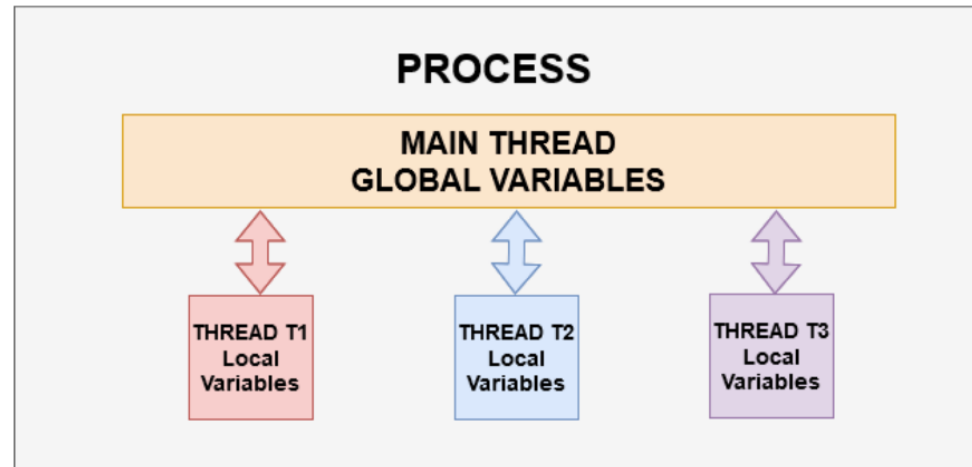
Shared Model

Process vs. Thread

- A **process** is an instance of a running program.
- **Process** may contain one or more **threads**, but a thread cannot contain a **process**.
- **Process** has a self-contained execution environment. It has its own memory space.
- Process don't share its memory
- Multiple **threads** in a **process** share resources.

Parallel Computation

Threading allows you to have different parts of your process run concurrently. For example, A web-browser could be a process, an application running multiple cameras simultaneously could be a process.



multiple threads work together to achieve a common goal

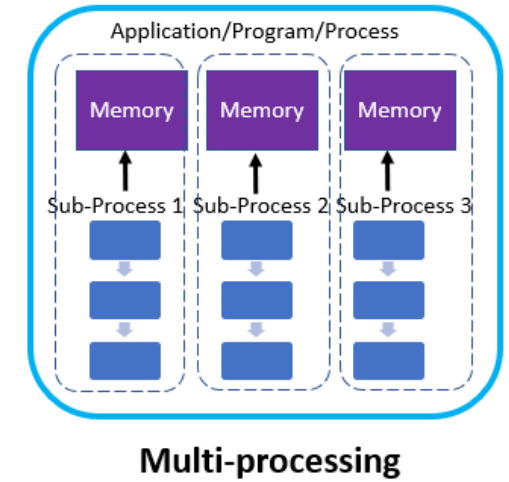
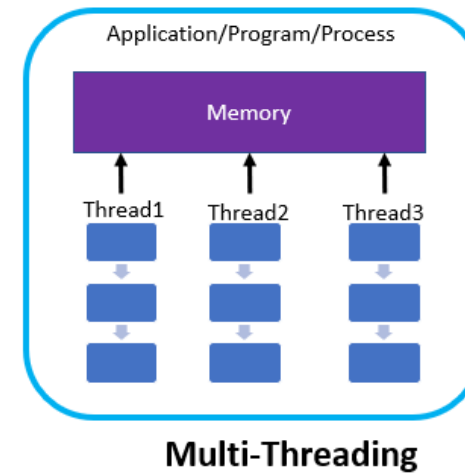
Parallel Computation

Multiprocessor/multicore

several processors work on data stored in shared memory

Cluster

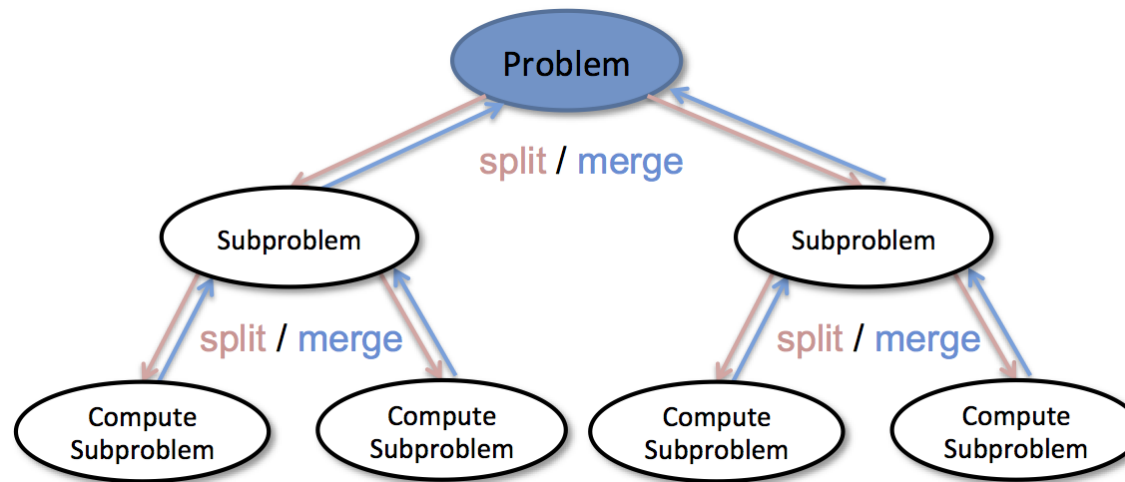
several processor units work together
by exchanging data over a network



Parallelism in Practice

Divide-Conquer-Combine as the central model

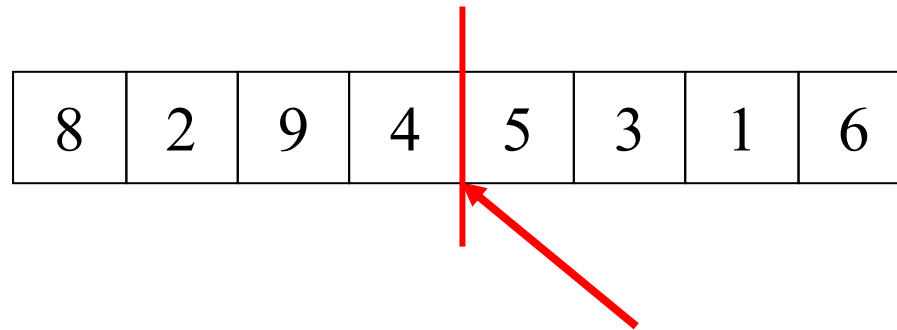
- **Divide:** partition data/workload.
- **Conquer:** same operation on each partition independently.
- **Combine:** merge partial outputs.



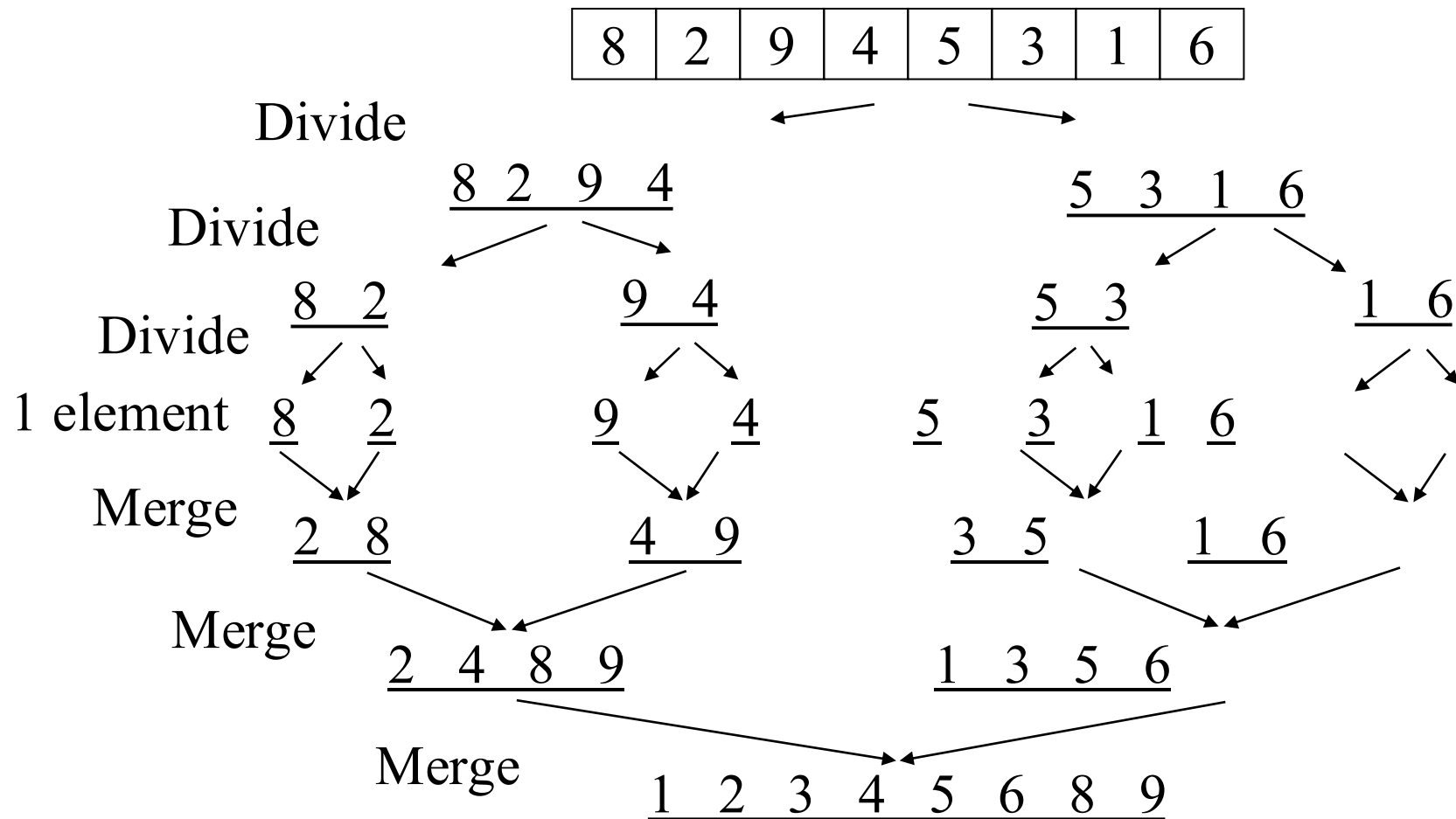
Example: Merge Sort

Merge Sort is a Divide and Conquer algorithm.

It divides the input array into two halves, calls itself for the two halves, and then merges the two sorted halves.



Example: Merge Sort



Example: Merge Sort

- Mergesort divides array in half and calls itself on the two halves.
- After returning, it merges both halves using a temporary array
- Each recursive call takes $2T\left(\frac{N}{2}\right)$ and merging takes $O(N)$
- Total of $O(n \log n)$